

CODEFORCES PERFORMANCE ANALYZER & RECOMMENDATION SYSTEM

Dataset Analysis Report

Prepared for: Machine Learning Course Project
Covers 4 datasets | 100+ features | Multi-table schema

2 Table 1: User Profile (user_profile.csv)

The user profile is the central entity of the dataset. Each row corresponds to one Codeforces user and aggregates their entire competitive programming history into a rich set of computed features. These span identity, contest performance, submission behavior, problem-solving patterns, and session characteristics.

2.1 Example Row

Example: user HackerSir
handle=HackerSir | cf_rating=1086 | cf_rank=newbie | total_contests=3 | total_submissions=22 | overall_ac_rate=0.682 | unique_problems_solved=15 | preferred_language=C++17 | preferred_time_of_day=afternoon | num_sessions=9

2.2 Feature Groups & Explanations

A. Identity & Profile

Feature	Data Type	Description
handle	string	Unique Codeforces username (primary key)
country / city / organization	string (nullable)	Geographic and institutional affiliation of the user
contribution	integer	Community contribution score (blog posts, comments, etc.)
friend_of_count	integer	Number of other users who have friended this user
registration_ts	Unix timestamp	Account registration timestamp in epoch seconds
registration_year	integer	Year the account was registered

B. Rating & Rank

Feature	Data Type	Description
cf_rating	integer	Current Codeforces rating
cf_max_rating	integer	All-time peak rating achieved
cf_rank	string	Current rank label (newbie, pupil, specialist, etc.)
cf_max_rank	string	Highest rank ever achieved
rating_bucket	categorical	Coarse rating grouping (e.g., 'newbie', '1000-1200')
rating_tier	categorical	Finer tier classification for ML stratification

C. Contest Performance

Feature	Data Type	Description
total_contests	integer	Total number of rated contests participated in
best_contest_rank / worst_contest_rank	integer	Extreme rank positions achieved in any single contest
avg_contest_rank / median_contest_rank	float	Central tendency measures for contest placement
total_rating_change	integer	Cumulative rating delta over all contests
avg_rating_change / std_rating_change	float	Mean and standard deviation of per-contest rating changes
max_positive_change / max_negative_change	integer	Largest single-contest gain and loss
num_positive_changes / num_negative_changes / num_zero_changes	integer	Count of contests resulting in gain, loss, or no change
improvement_rate	float	Ratio of positive-change contests to total contests
win_streak_max / loss_streak_max	integer	Longest consecutive positive or negative rating runs
contests_per_year	float	Average contest participation rate annualized
rating_volatility	float	Measure of rating stability / instability over time

D. Submission Statistics

Feature	Data Type	Description
total_submissions	integer	Total number of submissions across all problems
total_ac / total_wa / total_tle / total_mle / total_re / total_ce	integer	Count of each submission verdict type
overall_ac_rate / overall_wa_rate / overall_tle_rate	float [0,1]	Proportion of submissions receiving each verdict
avg_attempts_to_ac / median_attempts_to_ac	float	Average and median attempts needed to reach accepted verdict
problems_solved_first_try / first_try_rate	integer / float	Count and proportion of problems solved on the very first attempt

E. Problem-Solving Profile

Feature	Data Type	Description
unique_problems_tried / unique_problems_solved	integer	Total distinct problems attempted and solved

Feature	Data Type	Description
solve_rate	float [0,1]	Fraction of attempted problems eventually solved
avg_solved_rating / median_solved_rating / max_solved_rating / min_solved_rating / std_solved_rating	float	Distribution of difficulty ratings of solved problems
num_solved_unrated	integer	Problems solved that had no official difficulty rating
solved_lte_1000 ... solved_gt_3000	integer	Count of problems solved within each Codeforces rating band
solved_index_A/B/C/D/E_plus	integer	Count of solved problems by contest problem index (difficulty proxy)

F. Language Usage

Feature	Data Type	Description
preferred_language	string	Most frequently used programming language
preferred_language_family	categorical	Language family: C++, Python, Java, etc.
num_languages_used	integer	Total distinct programming languages used
pct_cpp / pct_python / pct_java	float [0,1]	Proportion of submissions in each major language

G. Account Activity & Temporal Behavior

Feature	Data Type	Description
first_submission_ts / last_submission_ts	Unix timestamp	Timestamps of very first and most recent submissions
account_age_days	float	Days since account registration to last submission
active_days_span	float	Days between first and last submission
subs_per_day / subs_per_active_day	float	Submission frequency over full life vs. active period only
distinct_active_days	integer	Number of unique calendar days with at least one submission
contest_sub_count / practice_sub_count / virtual_sub_count	integer	Submission counts split by context type
contest_sub_ratio / practice_sub_ratio	float [0,1]	Share of submissions coming from contest vs. practice mode
preferred_time_of_day	categorical	Peak activity period: morning, afternoon, evening, night
weekend_sub_ratio	float [0,1]	Proportion of submissions made on weekends

H. Tag & Session Aggregates

Feature	Data Type	Description
unique_tags_attempted / unique_tags_solved	integer	Breadth of topic categories engaged with
tag_coverage_pct	float [0,1]	Fraction of all known Codeforces tags the user has attempted
num_weak_tags / num_strong_tags	integer	Count of topics with low vs. high success rates
strongest_tag / weakest_tag	string	Topic with highest and lowest performance score
num_sessions	integer	Total number of distinct study sessions
avg_session_duration_min	float	Average length of a study session in minutes
avg_subs_per_session / avg_ac_per_session	float	Average productivity metrics per session
avg_time_between_subs_hours	float	Mean inter-submission gap — proxy for thinking time
avg_passedtests_on_wa	float	Average test cases passed even on wrong answer submissions

3 Table 2: User Tag Matrix (user_tag_matrix.csv)

This table provides a per-user, per-tag breakdown of submission performance. Each row represents one user's experience with one problem category (tag), making it a long-format pivot table ideal for collaborative filtering and tag-based recommendation.

3.1 Example Row

Example: HackerSir on 'binary search'
handle=HackerSir | tag=binary search | total_attempts=6 | ac_count=3 | wa_count=3 | ac_rate=0.50 | avg_rating_attempted=1550 | avg_rating_solved=1500 | max_rating_solved=1700 | weakness_score=0.973 | strength_score=0.693 | difficulty_push=-50

3.2 Feature Explanations

Feature	Data Type	Description
handle	string	Foreign key linking to user_profile
tag	string	Codeforces problem tag (e.g., binary search, dp, greedy)
total_attempts	integer	Total submissions for problems of this tag
ac_count / wa_count / tle_count / mle_count / re_count	integer	Submission verdicts for this specific tag
ac_rate / wa_rate / tle_rate	float [0,1]	Verdict proportions within this tag
avg_rating_attempted	float	Mean difficulty of all problems attempted in this tag
avg_rating_solved	float	Mean difficulty of problems actually solved in this tag
max_rating_solved / min_rating_solved	integer	Hardest and easiest problems solved within this tag
std_rating_solved	float	Spread/variability in difficulty of solved problems for this tag
weakness_score	float	Composite score indicating how weak the user is in this tag (higher = weaker)
strength_score	float	Composite score indicating how strong the user is in this tag (higher = stronger)
difficulty_push	float	Difference between avg rating solved and avg rating attempted; negative = struggling on harder problems
first_attempt_ts / last_attempt_ts	Unix timestamp	Time range of activity in this tag
days_active_in_tag	float	Number of days between first and last attempt in this tag
contest_attempts / practice_attempts	integer	Attempts broken down by contest vs. practice context

4 Table 3: User Rating History (user_rating_history.csv)

This table captures each user's contest-by-contest rating trajectory. It is a time series table that enables longitudinal analysis, trend modeling, and rating prediction tasks. Each row is one contest entry for one user.

4.1 Example Row

Example: HackerSir's first contest
handle=HackerSir | contest_index=1 | contest_id=1562 | contest_name=Codeforces Round 741 (Div.2) | rank=5249 | old_rating=0 | new_rating=465 | rating_change=+465 | is_positive_change=1 | running_max_rating=465 | cumulative_change=465 | contest_day_of_week=3

4.2 Feature Explanations

Feature	Data Type	Description
handle	string	Foreign key linking to user_profile
contest_index	integer	Sequential contest number for this user (1 = first ever contest)
contest_id	integer	Codeforces internal contest identifier
contest_name	string	Human-readable contest name
contest_ts	Unix timestamp	Contest start time in epoch seconds
contest_year / contest_month	integer	Calendar decomposition of contest date
rank	integer	User's final rank in the contest (lower is better)
old_rating / new_rating	integer	Rating before and after the contest
rating_change	integer	Signed delta: new_rating - old_rating
rating_change_abs	integer	Absolute value of rating_change
is_positive_change / is_negative_change	binary (0/1)	Indicator flags for contest outcome direction
running_max_rating / running_min_rating / running_avg_rating	float	Cumulative statistics up to and including this contest
rating_diff_from_peak	integer	How far the user is below their all-time peak at this moment
cumulative_change	integer	Running sum of all rating changes through this contest

Feature	Data Type	Description
contests_since_last_positive / contests_since_last_negative	integer	Streaks: contests elapsed since last improvement or decline
contest_day_of_week	integer (0=Mon)	Day of week the contest took place
rank_percentile_approx	float	Approximate percentile of the user's rank within the contest

5 Table 4: Study Sessions (study_session.csv)

The study session table models discrete problem-solving sessions, defined as periods of consecutive activity for a single user. This table is particularly valuable for understanding behavioral patterns, productivity cycles, and learning rhythms at a granular level.

5.1 Example Row

Example: HackerSir session #1

session_id=HackerSir_0 | date=2021-08-26 | start_hour=12 | duration_min=3.08 | num_submissions=3 | num_ac=1 | ac_rate=0.333 | avg_rating=800 | dominant_tag=brute force | is_practice_session=1 | num_wa_before_ac=1

5.2 Feature Explanations

Feature	Data Type	Description
handle	string	Foreign key linking to user_profile
session_id	string	Globally unique session identifier (handle_index format)
session_index	integer	Sequential session number for this user
start_ts / end_ts	Unix timestamp	Session start and end times
start_date / end_date	date string	Human-readable session dates
start_hour / end_hour	integer (0-23)	Hour of day the session started and ended
duration_min / duration_sec	float	Session length in minutes and seconds
day_of_week	integer (0=Mon)	Day of week the session fell on
is_weekend	binary (0/1)	Whether the session occurred on a Saturday or Sunday
time_of_day_start	categorical	Time-of-day label: morning, afternoon, evening, night
year / month	integer	Calendar decomposition of the session date
num_submissions	integer	Total submissions made during this session
num_ac / num_problems_tried / num_problems_solved	integer	Accepted verdicts and distinct problems worked on/solved
ac_rate	float [0,1]	Proportion of submissions that were accepted in this session
subs_per_hour / problems_per_hour	float	Productivity rates normalized to per-hour basis
avg_rating / max_rating / min_rating / std_rating	float	Difficulty profile of problems attempted in the session
dominant_tag	string	The most common problem tag in this session

Feature	Data Type	Description
num_unique_tags	integer	Number of distinct tags encountered during the session
languages_used	string (pipe-separated)	All programming languages used in this session
is_contest_session / is_practice_session	binary (0/1)	Flags indicating the session type
num_wa_before_ac	integer	Number of wrong answers before the first accepted submission

6 Potential Machine Learning Features

The following derived features can be engineered from the raw dataset to improve predictive modeling and recommendation quality. They are organized by the ML task they serve.

6.1 For Rating Prediction

- **Linear regression coefficient over the rating time series** — captures growth trend better than a snapshot. Rating trajectory slope:
- **Average number of contests required to recover from a negative change** — signals mental resilience. Rating recovery speed:
- **Standard deviation of inter-contest intervals** — regular practice correlates with steady improvement. Contest frequency regularity:
- **cf_rating / cf_max_rating** — measures how far a user has fallen from their personal best. Peak-to-current ratio:
- **Exponentially weighted average of the last N rating changes** — recent form is more predictive than all-time average. Weighted recent performance:

6.2 For Problem Recommendation

- **Per-user strength_score across all tags** — serves as a user embedding for collaborative filtering. Tag affinity vector:
- **Max rating solved per tag** — identifies the hardest problems in each category the user can handle. Tag difficulty frontier:
- **Days since last_attempt_ts per tag** — decays engagement scores to surface neglected topics. Tag recency weight:
- **For each tag, (avg_rating_attempted - avg_rating_solved)** — quantifies difficulty mismatch. Skill gap index:
- **Ratio of D/E problems to A/B problems solved** — measures readiness for harder problem types. Problem index progression:
- **Cosine similarity between user tag vectors** — enables user-based collaborative filtering. Cross-tag similarity:

6.3 For Learning Pattern Analysis

- **Linear trend of ac_rate or problems_per_hour over sequential sessions** — measures improvement within practice sessions. Session productivity trend:
- **Correlation between duration_min and ac_rate** — identifies each user's productive sweet spot. Optimal session length:

- **Average ac_rate grouped by time_of_day_start** — reveals when a user performs best. Time-of-day efficiency:
- **contest_sub_ratio** — users with balanced ratios often show more consistent improvement. Contest vs. practice balance score:
- **Moving average of first_try_rate over time** — improving first-try rate indicates mastery consolidation. First-try solve rate trend:
- **Active days per month normalized by account age** — high consistency correlates with rating growth. Study consistency score:

6.4 For User Segmentation / Clustering

- **[contest_sub_ratio, practice_sub_ratio, virtual_sub_ratio]** — classifies users as contest-focused, practice-focused, or balanced. Playstyle vector:
- **unique_tags_solved vs. avg solved problems per tag** — identifies generalists vs. specialists. Breadth vs. depth score:
- **One-hot or embedding of preferred_language_family** — useful as a demographic feature. Language preference encoding:
- **Max submissions in any single day / avg daily submissions** — identifies bursty vs. steady practitioners. Activity burst indicator:
- **Number of contests spent in each rating bucket before promotion** — signals learning speed. Rating band dwell time:

7 Potential Insights from the Dataset

7.1 User Performance Insights

- **High rating_volatility users may be aggressive risk-takers or inconsistent performers. Separating these clusters can refine recommendation strategies.** Rating volatility vs. growth:
- **improvement_rate directly measures how often a user gains rating. Users below 40% may be in a plateau and benefit from remedial tag-focused practice.** Improvement rate as a health signal:
- **Comparing contests_per_year against rating growth can reveal whether participating more frequently helps or harms performance at different skill levels.** Impact of contest frequency:
- **High first_try_rate suggests solid foundational understanding. Low first_try_rate with high solve_rate suggests trial-and-error — both are valid but call for different recommendation strategies.** First-try rate as a mastery indicator:

7.2 Learning Behavior Insights

- **Analyzing the correlation between avg_session_duration_min and avg_ac_per_session can reveal whether longer sessions have diminishing returns (fatigue) or compounding gains (flow states).** Session length and productivity:
- **Users may consistently perform better at certain times of day. Personalized scheduling nudges (e.g., 'your accuracy is 20% higher in the evening') could improve outcomes.** Preferred time and accuracy:
- **weekend_sub_ratio combined with session ac_rate can identify whether casual weekend practice is as productive as structured weekday sessions.** Weekend vs. weekday patterns:
- **Comparing ac_rate in contest sessions vs. practice sessions reveals whether high-pressure or low-pressure environments are more effective per user.** Practice context effectiveness:

7.3 Tag-Level Weakness Analysis

- **The weakness_score field in the tag matrix can be clustered to identify common struggle patterns (e.g., many users weak in dynamic programming, graph theory, or segment trees).** Weakness clusters:
- **Comparing max_rating_solved per tag against the user's cf_rating reveals whether they are punching above or below their weight in each category.** Difficulty ceiling per tag:

- **Tags with many days since last_attempt_ts and high weakness_score are prime candidates for targeted recommendation — the user has both need and recency gap.**Tag neglect detection:
- **For each tag, the ratio of practice_attempts to contest_attempts shows whether users are preparing appropriately for areas they encounter in contests.**Practice-to-contest transfer:

7.4 Recommendation System Insights

- **Combining avg_solved_rating and solve_rate by tag enables a zone-of-proximal-development approach — recommend problems slightly above the user's current comfort zone.**Optimal next problem difficulty:
- **For new users (few sessions, low total_submissions), registration_year, country, and organization can seed collaborative filtering before sufficient data is collected.**Cold-start handling:
- **Long gaps since last_submission_ts, declining subs_per_active_day, and declining improvement_rate may predict disengagement before the user churns from the platform.**Churn prediction signals:
- **Matching a user's current cf_rating and contest performance patterns with upcoming contest difficulty profiles can recommend which contest type (Div. 2, Div. 3, Educational) to attempt next.**Personalized contest targeting: